UNIVERSIDADE FEDERAL DE SERGIPE

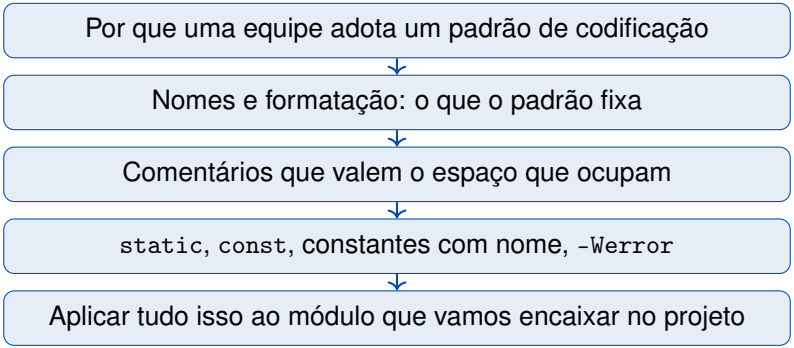
Padrões de codificação

E a prática de modularizar seguindo um padrão

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

O que você vai aprender hoje



O problema: um exemplo de código ruim que funciona

Onde estamos

Na aula passada o `notas.c` virou quatro arquivos: `matriz`, `estat`, `turma` e o `main`.

Hoje um colega manda o `ranking.c`: classifica os alunos pela média. Ele jura que funciona — e funciona.

O que vamos descobrir

Não tem `.h`: para chamá-lo, o `main.c` copiou as assinaturas na mão.

Guarda estado em variáveis globais.

Ninguém consegue ler sem parar para decifrar.

A pergunta de hoje

O que separa um código que *funciona* de um código que a turma toda consegue manter?

Um exemplo de código ruim

Exemplo de código ruim: o estado

```
double R[50];
int Q[50];
int N;

void troca(int a,int b){int t=Q[a];Q[a]=Q[b];Q[b]=t;}

int maior(double x,double y){ return x>y; }
```

Três variáveis globais e duas funções auxiliares que qualquer outro arquivo do projeto enxerga.

Exemplo de código ruim: o cálculo

```
void calcula(Turma *t, int flag)
{
  double tmp;
  N = t->nalunos;
  for(int i=0;i<N;i++){
  R[i]=media(t->notas[i],t->navaliacoes);
    Q[i]=i;                    /* guarda o indice i em Q[i] */
  }
    for(int i=0;i<N;i++)
        for(int j=0;j<N-1;j++)
            if(maior(R[Q[j+1]],R[Q[j]])) troca(j,j+1);
}
```

...e como o `main.c` conseguiu chamá-lo

```c
void mostra()
{
    printf("%-5s %-8s %8s %s\n","pos","aluno","media","situacao");
  for(int i=0;i<N;i++){
    printf("%-5d %-8d %8.2f %s\n", i+1, Q[i], R[Q[i]],
           R[Q[i]]>=7.0?"aprovado":"reprovado");
  }
}
```

```c
/* no main.c: o ranking nao tem header,
   copiei as assinaturas na mao */
void calcula(Turma *t, int flag);
void mostra();
```

O que vai acontecer?

Antes de rodar, decida

1. O projeto compila com `-Wall -Wextra`? Com quantos avisos?
2. A saída do ranking sai certa para os três alunos do `main.c`?
3. E se a turma tiver 60 alunos em vez de 3 — o que acontece?
4. Amanhã o `turma.c` ganha uma função chamada `troca`. O que acontece na ligação?

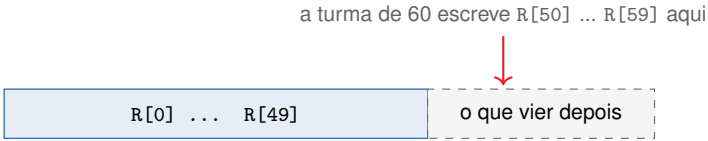
Anote suas quatro respostas antes de executar `make`.

O que aconteceu

```
$ make
ranking.c:16:10: warning: unused variable 'tmp' [-Wunused-variable]
ranking.c:14:28: warning: unused parameter 'flag' [-Wunused-parameter]
$ ./notas
pos   aluno       media situacao
1     2            9.25 aprovado
2     0            7.62 aprovado
3     1            5.75 reprovado
```

Perguntas 1 e 2 respondidas: compila com dois avisos, roda e acerta. As perguntas 3 e 4 a execução não mostrou — e são as que interessam.

Pergunta 3: o limite que ninguém escreveu



Em uma máquina depois de `R` vinha `Q`: os índices dos alunos viram lixo e as últimas colocações saem com média `0.00`.

Em outra depois de `R` vinha `N`: a contagem vira 50 e o programa imprime 50 linhas para uma turma de 60.

Não existe “o que acontece”: existe comportamento indefinido. O programa não quebrou — ele mentiu, e mentiu diferente em cada máquina. É o desfecho normal quando um limite fica implícito num número solto.

ISO/IEC 9899:2018, §J.2 (comportamento indefinido) [10]

Pergunta 4: dois `troca` no mesmo programa

```
$ gcc main.o turma.o matriz.o estat.o ranking.o outro.o -o notas
duplicate symbol '_troca' in: outro.o / ranking.o
ld: 1 duplicate symbols          # no Linux: "multiple definition of 'troca'"
```

O que aconteceu

Nada está errado no `outro.c`: ele só precisou de uma função `troca`. Quem reservou um nome comum para o projeto inteiro foi o `ranking.c`.

O custo

O erro aparece na *ligação*, longe do arquivo que o causou, e recai sobre quem chegou depois. Quanto maior o projeto, mais caro.

ISO/IEC 9899:2018, §6.2.2 (ligação externa e interna) [10]

Funciona não é o mesmo que está pronto

Quem lê esse código depois

Você daqui a três meses. O colega de equipe. Quem for consertar o bug às onze da noite.

Um trecho é escrito uma vez e lido dezenas — o tempo de leitura é o custo que importa.

O que um padrão de codificação é

Um conjunto de decisões tomadas *uma vez*, por escrito, para não serem rediscutidas a cada arquivo: nomes, formatação, comentários, organização dos arquivos.

Não é gosto pessoal: é acordo.

Kernighan & Pike (1999), *The Practice of Programming*, cap. 1 [11]

Nomes

Nomes dizem o que a coisa é

No código ruim	O que é de verdade	Nome
`R[i]`	média do aluno `i`	`media_aluno[i]`
`Q[i]`	quem está na *i*-ésima colocação	`posicao[i]`
`N`	quantos alunos entraram no ranking	`quantidade`
`calcula`	calcula *o quê*?	`ranking_calcula`
`mostra`	mostra *o quê*?	`ranking_imprime`
`flag`	nada: ninguém usa	—

Na Tarefa 2, quando o estado sai das globais e passa a viver numa `struct`, `ranking_calcula` vira `ranking_cria`: aí a função passa mesmo a *criar* alguma coisa.

Kernighan & Pike (1999), §1.1 “Names” [11]; McConnell (2004), cap. 11 [14]

A regra do escopo, e o prefixo do módulo

Escopo curto, nome curto

`i`, `j`, `n` em um laço de três linhas estão certos — todo mundo lê sem esforço.

Escopo longo, nome descritivo

O que é global, ou faz parte da interface, paga por um nome inteiro.

Prefixo de módulo

O projeto já tem a convenção:

`turma_cria`, `turma_libera`, `turma_relatorio`

O módulo novo entra igual:

`ranking_cria`, `ranking_libera`, `ranking_imprime`

C não tem *namespace*: o prefixo é o nosso.

Kernighan & Pike (1999), §1.1 [11]; GNU Coding Standards, §5.4 [8]

Formatação

O mesmo laço, depois do `clang-format`

Código ruim

```
for(int i=0;i<N;i++){
R[i]=media(t->notas[i],
  t->navaliacoes);
  Q[i]=i;
}
  for(int i=0;i<N;i++)
      for(int j=0;j<N-1;j++)
          if(maior(R[Q[j+1]],R[Q[j]]))
      troca(j,j+1);
```

Formatado

```
for (int i = 0; i < N; i++) {
    R[i] = media(t->notas[i],
                 t->navaliacoes);
    Q[i] = i;
}
for (int i = 0; i < N; i++)
    for (int j = 0; j < N - 1; j++)
        if (maior(R[Q[j+1]], R[Q[j]]))
            troca(j, j + 1);
```

Byte por byte, o mesmo programa: só mudou o tempo que você gasta para ter certeza disso. Repare no que *não* mudou — `R`, `Q`, `N` e `maior` continuam lá.

O que um padrão de formatação fixa

- indentação: 4 espaços, nunca tabulação (ou o contrário — mas um só);
- largura máxima de linha: 80 colunas;
- onde a chave de abertura fica: função em linha nova, bloco na mesma linha;
- espaço depois de `if`, `for`, `while`; nenhum antes do parêntese da chamada;
- uma declaração por linha;
- ordem dentro do `.c`: `include`s, constantes, tipos, funções `static`, funções da interface.

Linux kernel coding style, §1–§3 [19]; GNU Coding Standards, §5.1 [8]

Consistência vale mais do que preferência

Por que não adianta discutir qual é o melhor

Chave na mesma linha ou na linha de baixo? Dois espaços ou quatro? Não há resposta certa — há projetos que escolheram. O ganho vem de *todo o arquivo* seguir a mesma escolha, porque aí a formatação deixa de chamar atenção e você lê o que o código faz.

Regra prática

Entrou em código alheio, siga o estilo dele. Um arquivo com dois estilos é pior do que um arquivo com o estilo que você não gosta.

Kernighan & Pike (1999), §1.3 “Consistency and Idioms” [11]

Comentários

Três comentários e o que cada um vale

```
Q[i] = i;                    /* guarda o indice i em Q[i] */
```

Repete o código. Se a linha mudar, ele mente.

```
/* ordena por media decrescente (usa selecao) */
... insercao ...
```

Desatualizado. Pior que nenhum: o leitor acredita nele.

```
/* Insercao, e nao um algoritmo melhor, porque
   uma turma cabe em uma sala. */
```

Explica a decisão — o que o código não consegue dizer sozinho.

McConnell (2004), cap. 32 [14]; Kernighan & Pike (1999), §1.6 [11]

Onde cada comentário mora

No `.h`: o contrato

Para quem vai *usar* sem abrir o `.c`:

- o que a função faz;
- o que devolve, inclusive no erro;
- de quem é a responsabilidade de liberar.

No `.c`: o porquê

Para quem vai *manter*:

- por que este algoritmo e não outro;
- que caso estranho aquela linha resolve;
- que suposição está sendo feita.

O comentário que descreve *como* costuma ser sintoma: quase sempre dá para trocá-lo por um nome melhor ou por uma função à parte.

McConnell (2004), cap. 32 [14]; GNU Coding Standards, §5.2 [8]

Sua vez: dois minutos, em dupla

```c
int p(double *v, int n)
{
    double s = 0;               /* soma */
    for (int i = 0; i < n; i++)
        s += v[i];
    if (s / n >= 7.0)           /* compara com 7.0 */
        return 1;
    return 0;
}
```

Esta função já está formatada no padrão — e mesmo assim três coisas dela quebram o que combinamos até agora. Quais? Escreva o nome que você daria à função, aos parâmetros e ao `7.0`.

O que dava para melhorar

1. Nomes de escopo curto num escopo longo
`p` e `v` estão na interface:
`aprovado_pela_media(const double *notas, int quantidade)`.

2. Comentários que repetem o código
“soma” e “compara com 7.0” somem. Falta o que o código não diz: o que a função devolve, e o que vale quando `n` é 0.

3. Número mágico
O `7.0` é regra da universidade, não da função: `NOTA_CORTE`, declarada uma vez.

De brinde
`const` declara que a função não mexe nas notas — e `s / n` com `n` igual a 0 é a pergunta que o comentário bom teria levantado.

Kernighan & Pike (1999), §1.1 e §1.5 [11]; McConnell (2004), cap. 11 [14]

O que o compilador cobra

Número mágico vira constante com nome

Código ruim

```
double R[50];
int Q[50];
...
R[Q[i]] >= 7.0 ? "aprovado"
               : "reprovado"
```

O 50 é um limite escondido; o 7.0 é uma regra da universidade espalhada pelo código.

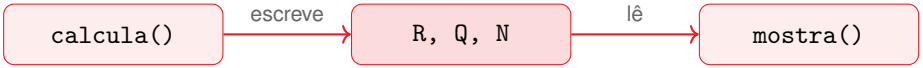
No padrão

```
/* em ranking.h */
#define RANKING_NOTA_CORTE 7.0
```

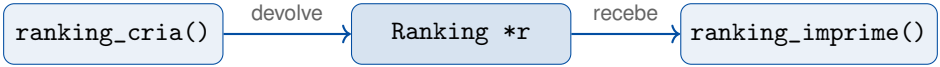
O 50 simplesmente desaparece: o ranking aloca o tamanho da turma, como já fazemos desde a aula de alocação dinâmica.

Kernighan & Pike (1999), §1.5 “Magic Numbers” [11]

Variável global: o estado que ninguém vê passar



a ligação entre as duas não aparece em assinatura nenhuma



a ligação está escrita no protótipo — o compilador cobra

Com globais, chamar `mostra()` antes de `calcula()` imprime lixo — e nada no código avisa. Com o parâmetro, não existe ranking para imprimir antes de alguém criá-lo.

Kernighan & Pike (1999), §4.5 “Interface Principles” [11]; McConnell (2004), §13.3 [14]

`static`: o que é interno não vaza

```c
static void ordena(Ranking *r) { ... }   /* so ranking.c enxerga */
Ranking *ranking_cria(const Turma *t) { ... }  /* interface */
```

Sem `static`

`troca` e `maior` entram na tabela de símbolos do `.o`. Outro módulo com uma `troca` própria derruba a ligação com símbolo duplicado.

Com `static`

O símbolo não sai do arquivo. A regra fica simples: *está no `.h`? então é público. Não está? então é `static`.*

ISO/IEC 9899:2018, §6.2.2 [10]; Kernighan & Ritchie (1989), §4.6 [12]

`const`: a promessa de não mexer

```c
Ranking *ranking_cria(const Turma *t);   /* le a turma, nao altera */
void ranking_imprime(const Ranking *r);  /* so imprime */
```

Para quem lê
O protótipo já responde “essa função estraga meus dados?” sem abrir o `.c`.

Para quem escreve
Se a implementação tentar escrever, o compilador reclama. A promessa é verificada, não confiada.

Já usamos isso sem comentar: `media(const double *v, int n)` e `turma_relatorio(const Turma *t)` estão assim desde a aula de modularização.

O padrão que a máquina cobra

```
CFLAGS = -std=c17 -Wall -Wextra -Werror -pedantic -g
```

`-Wall -Wextra`: liga os avisos úteis.

`-Werror`: aviso vira erro — o projeto não compila com pendência.

`-pedantic -std=c17`: recusa extensões que só o seu compilador entende.

Por que já no primeiro dia
Aviso que ninguém corrige some no meio de outros trinta. Com `-Werror` a dívida nunca acumula: ou o aviso é real e você corrige, ou não é e você escreve por que não é.

E há quem discorde: o GNU Coding Standards diz que "*the compiler should be your servant, not your master*" e deixa o `-Wall` a critério de cada um. Nenhum dos dois lados está errado — por isso a decisão é da equipe, e a nossa está escrita no `CFLAGS`.

GCC Manual, "Options to Request or Suppress Warnings" [9]; GNU Coding Standards, §5.3 [8]

Recapitulando: o seu checklist de revisão

☐ todo nome público começa com `ranking_`?

☐ nome de uma letra só dentro de laço curto?

☐ nenhuma variável global — o estado viaja por parâmetro?

☐ nenhum número solto: o 50 sumiu e o 7.0 tem nome?

☐ tudo que não está no `.h` é `static`?

☐ `const` em todo parâmetro que a função só lê?

☐ cada função do `.h` diz o que devolve no erro e quem libera a memória?

☐ o arquivo inteiro tem uma formatação só?

Cada linha deste checklist é uma decisão que a equipe tomou uma vez para não rediscutir a cada arquivo.

Ferramentas

Escrever o padrão em um arquivo

```
# .clang-format --- na raiz do projeto
BasedOnStyle: LLVM
IndentWidth: 4
UseTab: Never
ColumnLimit: 80
BreakBeforeBraces: Linux
AllowShortIfStatementsOnASingleLine: false
```

```
formato:
    clang-format -i *.c *.h
```

Com o padrão em arquivo, ninguém precisa lembrar dele — e a revisão de código deixa de gastar tempo com indentação.

LLVM Project, *ClangFormat Style Options* [20]

O que a ferramenta faz, e o que ela não faz

Ela resolve

- indentação, espaços, chaves;
- largura de linha e quebras;
- ordem dos `includes`, se você pedir.

Ela não resolve

- `R`, `Q` e `N` continuam com esses nomes;
- a global continua global;
- o módulo continua sem `.h`.

`clang-format` cuida da parte mecânica do padrão. A parte que exige entender o programa — nomes, fronteiras, responsabilidades — continua sendo sua.

LLVM Project, *ClangFormat* [20]; Kernighan & Pike (1999), §1.3 [11]

Mãos à obra

O `ranking.h` que faltava

```c
#ifndef RANKING_H
#define RANKING_H
#include "turma.h"
#define RANKING_NOTA_CORTE 7.0
typedef struct {
    int    *posicao;   /* posicao[0] = indice do aluno de maior media */
    double *media;     /* media[i] = media do aluno de indice i */
    int     quantidade;
} Ranking;

/* Devolve NULL se faltar memoria; quem chama libera. */
Ranking *ranking_cria(const Turma *t);
void     ranking_libera(Ranking *r);
int      ranking_aprovado(const Ranking *r, int colocacao);  /* 0 = 1o */
void     ranking_imprime(const Ranking *r);
#endif /* RANKING_H */
```

Altere o programa

Tarefa 1

agora, em aula

Rode `clang-format -i ranking.c` e leia o `diff`. Depois renomeie `R`, `Q`, `N`, `calcula` e `mostra`; apague o `flag`.

Tarefa 2

começa em aula, termina em casa

Escreva o `ranking.h` com o contrato, tire as globais, marque `static` o que não está no header e apague as assinaturas copiadas no `main.c`.

Tarefa 3

em casa

Acrescente `-Werror -pedantic` ao `CFLAGS` e corrija as dependências de `ranking.o` no `Makefile`.

Critério de pronto: compila sem um aviso, `nm ranking.o` só com a interface, saída igual à do começo da aula.

Desafio

Um critério de ordenação que o cliente escolhe

em casa

Hoje o ranking ordena por média. Faça o critério virar parte da interface, sem que `turma.c` ou `estat.c` sejam tocados:

- acrescente ao `ranking.h` um tipo de critério — por média, por maior nota, por menor desvio;
- `ranking_cria` recebe o critério; a função de comparação correspondente fica `static` dentro de `ranking.c`;
- documente no header o que cada critério faz e o que acontece em caso de empate;
- compile com `-Werror` e mostre que nenhum símbolo novo vazou: confira com `nm ranking.o`.

Fechamento

O que mudou hoje

O programa imprime exatamente o mesmo que imprimia no começo da aula. Mudou quem consegue mexer nele sem medo.

O fio da aula

Nome é documentação, formatação é acordo, comentário é o porquê, `static` e `const` são promessas verificadas e `-Werror` é quem cobra.

Para levar

- padrão de codificação não é gosto: é decisão tomada uma vez para não ser rediscutida;
- a ferramenta formata; separar responsabilidades continua sendo trabalho seu;
- código que só você entende é código que só você mantém.

Referências I

André Backes.
Linguagem C: completa e descomplicada.
Elsevier, Rio de Janeiro, 2013.

Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2015.
Seção 3.9: alinhamento de dados e layout de structs.

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: teoria e prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.

Referências II


Paul Deitel and Harvey Deitel.
C: como programar.
Pearson Prentice Hall, São Paulo, 6 edition, 2011.


Ulrich Drepper.
What Every Programmer Should Know About Memory.
Red Hat, Inc., 2007.
Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.


Ulrich Drepper.
How to write shared libraries.
Documento técnico, 2011.
Versão 4.1.2.

Referências III


Paulo Feofiloff.
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.


Free Software Foundation.
Gnu coding standards.
Free Software Foundation.
Acesso em 2026.

Referências IV


Free Software Foundation.
Using the GNU compiler collection — options to request or suppress warnings.
Manual do GCC.
Acesso em 2026.


ISO/IEC.
Iso/iec 9899:2018 — information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seção 6.7.2.1: structure and union specifiers.

Referências V


Brian W. Kernighan and Rob Pike.
The Practice of Programming.
Addison-Wesley, Reading, MA, 1999.
Cap. 1, “Style”: nomes, expressões, consistência, números mágicos e comentários.


Brian W. Kernighan and Dennis M. Ritchie.
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.

Referências VI


K. N. King.
C Programming: A Modern Approach.
W. W. Norton, New York, 2 edition, 2008.
Capítulo 16: estruturas, uniões e enumerações.


Steve McConnell.
Code Complete: A Practical Handbook of Software Construction.
Microsoft Press, Redmond, WA, 2 edition, 2004.
Cap. 11 (nomes de variáveis), cap. 31 (layout e estilo) e cap. 32 (código autodocumentado).

Referências VII


Robert Mecklenburg.
Managing Projects with GNU Make.
O'Reilly Media, 3 edition, 2004.


Herbert Schildt.
C completo e total.
Makron Books, São Paulo, 3 edition, 1997.


Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.
Ferramenta usada no tutorial para detectar vazamentos e uso após `free`.

Referências VIII


Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.


The Linux Kernel Developers.
Linux kernel coding style.
Documentação do kernel Linux, `Documentation/process/coding-style.rst`.
Acesso em 2026.


The LLVM Project.
Clang-format style options.
Documentação do Clang.
Acesso em 2026.

Referências IX


Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.
Dynamic storage allocation: A survey and critical review.
In *International Workshop on Memory Management (IWMM)*, pages 1–116. Springer, 1995.
Panorama de estratégias de alocação dinâmica e fragmentação.


Nivio Ziviani.
Projeto de algoritmos: com implementações em Pascal e C.
Cengage Learning, São Paulo, 3 edition, 2011.